

LazyDP: Co-Designing Algorithm-Software for Scalable Training of Differentially Private Recommendation Models

Juntaek Lim
KAIST
juntaek0425@kaist.ac.kr

Kiwan Maeng
Pennsylvania State University
kvm6242@psu.edu

Youngeun Kwon
KAIST
yekwon@kaist.ac.kr

G. Edward Suh
FAIR at Meta
Cornell University
edsuh@meta.com

Ranggi Hwang
KAIST
ranggi.hwang@kaist.ac.kr

Minsoo Rhu
KAIST
mrhu@kaist.ac.kr

Abstract

Differential privacy (DP) is widely being employed in the industry as a practical standard for privacy protection. While private training of computer vision or natural language processing applications has been studied extensively, the computational challenges of training of recommender systems (RecSys) with DP have not been explored. In this work, we first present our detailed characterization of private RecSys training using DP-SGD, root-causing its several performance bottlenecks. Specifically, we identify DP-SGD’s noise sampling and noisy gradient update stage to suffer from a severe compute and memory bandwidth limitation, respectively, causing significant performance overhead in training private RecSys. Based on these findings, we propose LazyDP, an algorithm-software co-design that addresses the compute and memory challenges of training RecSys with DP-SGD. Compared to a state-of-the-art DP-SGD training system, we demonstrate that LazyDP provides an average 119× training throughput improvement while also ensuring mathematically equivalent, differentially private RecSys models to be trained.

ACM Reference Format:

Juntaek Lim, Youngeun Kwon, Ranggi Hwang, Kiwan Maeng, G. Edward Suh, and Minsoo Rhu. 2024. LazyDP: Co-Designing Algorithm-Software for Scalable Training of Differentially Private Recommendation Models. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’24)*, April 27-May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3620665.3640384>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ASPLOS ’24, April 27-May 1, 2024, La Jolla, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0385-0/24/04...\$15.00

<https://doi.org/10.1145/3620665.3640384>

1 Introduction

With recent advances in deep neural networks (DNNs), hyperscalers leverage DNN-based recommender systems (RecSys) to power their ads recommendation service [4, 9, 11, 19, 27, 51, 67]. The widespread adoption of RecSys, however, is raising serious concerns on protecting the *privacy* of users. This is because the most popular machine learning (ML) models utilized in personalized ads are trained over private user data (e.g., websites a user has visited, items a user has purchased). With increasing emphasis on privacy-preserving machine learning (PPML), *differential privacy* (DP) is gaining momentum as a well accepted notion of privacy and is widely being deployed in industrial applications, particularly for training ML algorithms requiring provable privacy guarantees [3, 6, 14, 59].

One of the most widely adopted *private* ML training algorithm is DP-SGD (differentially private stochastic gradient descent [1]), which is an extension to *non-private* SGD [39]. There are three important steps in DP-SGD that distinguish it from SGD: (1) unlike SGD which derives *per-batch* gradient during backpropagation, DP-SGD requires the derivation of *per-example* gradient, (2) to limit the influence of each training example, the per-example gradient is norm-clipped when the L2 norm of the gradient exceeds a predefined threshold, and (3) to guarantee DP, the averaged gradient is added with *Gaussian noise* before the gradient is used to update model weights (Section 2.4). With the recent surge of interest in PPML, there has been several prior work exploring DP-SGD in terms of its privacy, model accuracy, and its training throughput [2, 6, 12, 13, 25, 29, 36, 40, 42, 52, 54, 55, 66]. These studies, however, primarily focused on computer vision [12, 25, 36] or natural language processing [2, 6, 29, 42, 55, 66], so there is a dearth of prior work providing a systematic evaluation of DP-SGD’s computational challenges for RecSys [13, 52].

Given this landscape, an important motivation and key contribution of this work is the characterization and analysis on private RecSys training using DP-SGD, root-causing its bottlenecks from a systems perspective. Modern RecSys employs categorical features to enhance accuracy, which is

modeled using embedding layers. An embedding layer utilizes an *embedding table* (which is an array of embedding vectors) and *gathers* multiple embedding vectors to translate categorical features into embeddings. A unique property of embedding layers is that accesses to these tables are extremely “sparse”, e.g., embedding layer in MLPerf DLRM [45] accesses only 0.03% of the table content per each training iteration. Because the embedding tables are also model weights subject for training, an embedding layer trained using non-private SGD involves a sparse *gradient update* operation, targeting gradient updates only to the table locations storing the embeddings gathered during forward propagation.

Training embeddings with DP-SGD, on the other hand, requires the addition of Gaussian noise to *all* the embeddings within the embedding table, regardless of whether they were subject for embedding gathers or not during forward propagation. This in turn transforms SGD’s sparse gradient update into a dense “noisy” *gradient update* operation that modifies the *entire* embedding table. Our characterization uncovers the following two critical system-level challenges of DP-SGD’s noisy gradient update operation (Section 4.3).

- **Noise sampling.** Adding Gaussian noise to embedding vectors requires the sampling of a noise value from a normal distribution. Because noise sampling is required for every single embedding table entry and each sampling involves a series of trigonometric and logarithmic operations, noise sampling incurs *high computational overheads*.
- **Noisy gradient update.** As the noisy gradient is a dense tensor sized identically to the embedding table, noisy gradient update exhibits a *memory bandwidth limited behavior*. For large table sizes, which is common in modern RecSys models [46, 47, 63, 67], the noisy gradient update operation incurs severe performance overheads.

In this work, we present our algorithm-software co-design *LazyDP* that enables scalable and high-throughput DP-SGD training with privacy guarantees for RecSys. LazyDP can protect an arbitrary training example against an adversary who has access and control over the final trained model and all the other training data. While the adversary LazyDP assumes is slightly weaker than the original DP-SGD, our proposal provides the same DP-SGD level of privacy protection from most of the practical adversaries (Section 3 details our assumed threat model). LazyDP is based on the following key contributions that effectively addresses the compute (noise sampling) and memory (noisy gradient update) bottlenecks of private embedding layer training.

- **Lazy noise update.** Due to the sparse access nature of embedding gathers, the majority of embeddings are not accessed at any given training iteration, only updated using the noise but not the gradient. Generalizing this observation, if an embedding has not

been accessed during the *current* training iteration, it is most likely not going to be accessed again during the *next* iteration (and the iteration after that), only going through a series of noise updates without any accesses. Given such property, unless an embedding is scheduled to be accessed in its next training iteration, our proposal “delays” the noise update process for the embeddings at a given training iteration. Such optimization helps alleviate the memory bandwidth bottleneck of noisy gradient updates as the majority of noise update operations can be skipped (delayed), significantly reducing memory traffic. To make sure our lazy noise update algorithm does not affect the final trained model, i.e., the privacy of baseline DP-SGD, LazyDP ensures that all delayed noise updates are properly conducted *before* an embedding is actually accessed (Section 5.2.1).

- **Aggregated noise sampling.** While lazily adding noise to the embedding table helps reduce memory traffic incurred with noise updates, the compute bottlenecks of the noise sampling itself still remain. This is because the embedding that delayed its noise addition over N consecutive training iterations must eventually be accumulated with N separate noise values, each sampled from a normal distribution and causing identical noise sampling overhead vs. baseline DP-SGD. LazyDP exploits the mathematical principles behind normally distributed random variables and proposes “aggregated” noise sampling, a novel noise sampling algorithm which dramatically reduces the compute overheads of noise sampling by orders of magnitude (Section 5.2.2).

Overall, LazyDP provides an average 119× training speedup over representative RecSys models while guaranteeing mathematically equivalent, differentially private RecSys models to be trained vs. baseline DP-SGD.

2 Background

2.1 Recommendation Models

Modern RecSys typically combines two key components, a sparse embedding layer and a dense DNN layer with multi-layer perceptions (MLP). Such model architecture enables both sparse categorical features as well as dense continuous features to be captured, enhancing the model’s accuracy. In order to handle sparse categorical features, the embedding layer utilizes *embedding tables* to convert a categorical feature into an *embedding vector* (Figure 1). An embedding table is an array of embedding vectors and is indexed using a sparse feature input, which is a discrete valued index ID. As there can be millions to billions of items that can fall under a feature category (e.g., all videos available on Netflix), an embedding table can be sized at several tens to

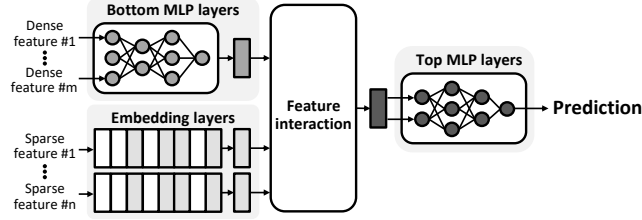


Figure 1. A RecSys model architecture using embedding layers.

hundreds of GBs, with recent large-scale RecSys even reaching TB-scale [46, 47, 63, 67]. When predicting a RecSys model output, multiple embedding vectors can be *gathered* from the embedding table, all of which are *pooled* into a single vector using a reduction operation, e.g., element-wise addition or multiplication.

Embedding layers have two distinct characteristics compared to dense DNN layers. First, they exhibit extremely low compute intensity because both embedding gather and pooling operations are limited by *memory bandwidth*. Secondly, embedding layers require *high memory capacity* due to the sheer size of the embedding tables, which need to encompass all categorical information as embedding vectors.

2.2 System Architecture for Training RecSys

The size of embedding tables in recent RecSys reached several hundreds of GBs to even TB-scale. As ML training is a throughput-oriented algorithm, system architectures for RecSys training typically employ throughput-optimized GPUs which employ bandwidth-optimized memory based on stacked DRAM (e.g., HBM). These high-bandwidth memory solutions, however, come with limited capacity, only providing several tens of GBs which makes it impossible to store the memory hungry embedding tables locally within the GPU.

To this end, state-of-the-art system architectures for training RecSys commonly employ a CPU-GPU based system [20, 37, 38, 46, 67]. Under such system design, the CPU is provisioned with a large pool of capacity-optimized DIMMs (e.g., LR-DIMM) to locally store large embedding tables, enabling the training of embedding layers using the CPU. The training of compute-intensive DNN layers is handed over to the GPU to best exploit its high compute and memory throughput. In the rest of this paper, we assume such CPU-GPU based system as the baseline system to train RecSys.

2.3 Differential Privacy

As ML applications have become more popular and widespread, the need to protect a user's data privacy has also grown significantly. This is because the training dataset used in ML is typically collected using crowdsourcing and may include sensitive private information of users. Given such vulnerability, several prior work demonstrated that an adversary that has only black-box access to the ML model can restore

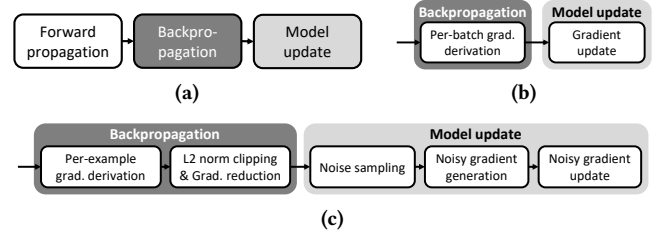


Figure 2. Non-private SGD vs. private SGD. (a) SGD and DP-SGD both go through the same set of operations during all of forward propagation and the activation gradient derivation of backpropagation. Key differences between the backpropagation and model update of (b) SGD and (c) DP-SGD include the L2 norm clipping, noise sampling, and updating the model weights with the noisy gradient.

individual training examples through training data extraction, leaking private information [7, 8, 22–24, 28, 48, 60].

Given this landscape, differential privacy (DP) has received a lot of attention in recent years as a well-accepted notion of privacy thanks to its strong and mathematically rigorous guarantees on privacy protection [15]. For an ML model to be differentially private, the trained model should be indistinguishable whether or not a particular training example was utilized to train the ML model, allowing the privacy of individual training examples to be protected with DP. In [1], Abadi et al. presented their seminal work on training DNN based ML algorithms using DP, aka DP-SGD. In the following subsection, we explain both non-private SGD and private DP-SGD training and discuss their key differences. For the interested readers, we refer to [1, 15–17] for a more detailed discussion on the mathematical foundation behind DP.

2.4 Non-private SGD vs. Private DP-SGD Training

Training an ML model requires updating its model weights through forward, backward propagation (i.e., backpropagation), and model update. Both SGD and DP-SGD involve an identical set of layer execution during forward propagation (Figure 2(a)). As for backpropagation, SGD and DP-SGD go through the same set of procedures to derive each layer's *activation gradient* but significantly differ in the way the *weight gradient* is derived. We summarize their differences below.

Non-private SGD. To update the ML model weights, the weight gradient must first be derived at a size identical to the target model size, which is subsequently utilized by the optimizer to train the model. A non-private SGD does this by *aggregating* the weight gradients derived for individual input examples that constitute an input mini-batch, effectively *reducing* them down into a single set of weight gradient (Figure 2(b)), the size of which becomes identical to the model weight size. In this paper, we refer to SGD's weight gradient

as the *per-batch* weight gradient to differentiate it from DP-SGD's *per-example* gradient as explained below.

Private DP-SGD. A key ingredient of DP-SGD [1] is to *clip* and add *noise* to the weight gradients during updates. As illustrated in Figure 2(c), rather than computing an averaged per-batch gradient, DP-SGD first derives the gradient for each *individual example* within a mini-batch. These *per-example* gradients are first clipped using a predetermined threshold, using L2 norm. After clipping, all the per-example gradients are averaged, and Gaussian noise is added. The final “noisy” gradient is used to update the model.

2.5 Related Work

Fast and memory-efficient DP-SGD. A key challenge with DP-SGD is that the derivation of per-example weight gradients requires separate memory allocations for each per-example weight gradient across all the layers, causing a memory capacity bottleneck (e.g., a mini-batch size of N requires N times larger memory allocations for storing weight gradient tensors in DP-SGD). To alleviate DP-SGD's memory capacity problem, Lee et al. [40] proposed *reweighted* DP-SGD (henceforth referred to as DP-SGD(R)) which reduces the memory allocation size of weight gradients at the expense of additional computation overheads. Specifically, DP-SGD(R) goes through the weight gradient derivation computation *twice*, (1) first computing the per-example weight gradient but only to derive the L2 norm clipping threshold values, which enables DP-SGD(R) to not have to incur high memory allocation overheads of the original DP-SGD, and then (2) utilizing these L2 norm values, DP-SGD(R) conducts a *reweighted* version of per-batch weight gradient derivation to derive the final aggregated weight gradient values, i.e., per-batch weight gradient.

While DP-SGD(R)'s need to go over both per-example and per-batch weight gradient derivation may seem to lead to worse performance than the original DP-SGD, DP-SGD(R) opens up unique opportunities to *fuse* several key bottleneck layers, achieving higher performance while also incurring smaller memory allocation size than DP-SGD [13, 54]. Additionally, the output model trained with DP-SGD(R) is mathematically identical to the original memory hungry DP-SGD algorithm. Because of DP-SGD(R)'s higher performance and smaller memory allocation while also guaranteeing the original DP-SGD's privacy protection, we consider DP-SGD(R) as a stronger baseline DP-SGD algorithm for characterization in Section 4. Recent work by Denison et al. [13] proposed a performance-optimized version of DP-SGD(R) (referred to as DP-SGD(F) in the rest of this paper) which is based on observations that RecSys only consists of embedding layers and linear MLP layers, allowing for the efficient estimation of the L2 norm of per-example gradient using standard backpropagation, without the need for deriving per-example weight gradient. As we detail in Section 4.1, DP-SGD(F) consistently provides higher performance than DP-SGD(R), so we assume

DP-SGD(F) as the baseline DP-SGD algorithm to compare against LazyDP in our final evaluation in Section 7.

Other than these closely related prior work, DiVa [54] is a recently proposed accelerator architecture for high-performance DP-SGD training. Unlike LazyDP which improves the performance of private sparse embedding layer training, DiVa focuses on optimizing the performance of *non-embedding* layers, only targeting computer vision and natural language processing ML algorithms. As such, DiVa suffers from the exact same compute and memory bottlenecks of RecSys training that GPU systems currently encounter upon, one which we detail in Section 4. In general, the key contribution of LazyDP is orthogonal to DiVa as our proposal complements DiVa's limitation by broadening its applicability for RecSys.

DP-SGD for training RecSys. Denison et al. [13] presents a privacy-utility trade-off analysis of employing DP-SGD for training real world ads dataset, demonstrating that DP-SGD can provide both privacy and good model accuracy for RecSys. Similar to LazyDP, EANA [52] presents an alternative DP-SGD algorithm to alleviate the compute and memory overheads of training RecSys embedding tables, improving overall training throughput. Unlike LazyDP, however, EANA's privacy protection is fundamentally weaker because it modifies the original DP-SGD algorithm to add noise *only* to the accessed embedding vectors, weakening DP-SGD's ability to anonymize individual training example. As an intuitive example, EANA never adds noise to an embedding vector if it has never been accessed, which will directly leak the fact that no user data contains the corresponding feature – the original DP-SGD algorithm does not leak such information because it adds noise to *all* the embedding vectors regardless of whether they have been accessed. LazyDP does not alter the mathematical foundation behind the baseline DP-SGD algorithm and enjoys the same level of privacy guarantee, while significantly alleviating its performance overhead. In Section 7.4, we compare LazyDP against EANA to demonstrate LazyDP's merits.

3 Threat Model

LazyDP protects an arbitrary training example against an adversary who has access and control over (1) the final trained model and (2) all the other training data except for the specific example under attack. The adversary that LazyDP assumes, while slightly weaker than what the original DP-SGD can handle, is still highly relevant to practical scenarios.

The original DP-SGD can protect against an adversary with an access and control over (1) the final trained model, (2) all the other training data, and additionally, (3) *all the intermediate states* of the model (i.e., all the gradients) during the training process [1]. This original threat model of DP-SGD is often considered excessively strong [50], as many real-world adversaries can only access the final trained model, either

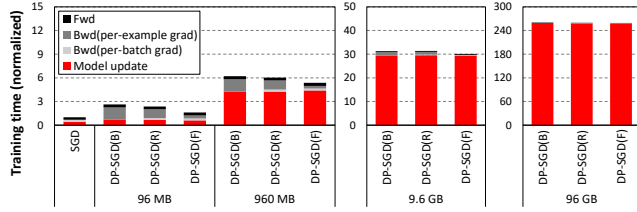


Figure 3. Breakdown of SGD and DP-SGD’s training time into key stages of forward and backward propagation. SGD’s training time remains almost constant regardless of the table size, so this figure only shows a single SGD data point under the default 96 GB for brevity. All data points are normalized to this SGD (the leftmost bar).

directly or through an inference API, but cannot access the intermediate gradients. For example, recent real-world attacks that successfully extracted training data from a released GPT-2 model [8], released Stable Diffusion model [7], and by querying ChatGPT API [48], were all done only with the access to the final trained model and not the intermediate gradients. When considering these practical adversaries, LazyDP provides the exact same privacy guarantee as the original DP-SGD. The same weaker but practical threat model was assumed in many other prior work as well [10, 21, 49, 50, 52, 61] including EANA [52]. In fact, the reason for the overly strong threat model of DP-SGD (that does not match real-world attackers’ capability) is due to the current limitation in privacy accounting. So far, there is no known mathematical technique to directly analyze privacy against the more practical adversaries that cannot access the intermediate gradients [50].

Admittedly, there are cases where the adversary can access the gradients. Such cases include federated learning [44], where the gradients of the individual trainer are sent to an untrusted server. LazyDP can also be vulnerable in special cases where the parameter server in a distributed training is considered untrusted [57] or uses a specialized hardware [43, 58] that may be hacked. LazyDP cannot provide protection that matches the original DP-SGD in these cases.

4 Workload Characterization on Private RecSys Training with DP-SGD

This section takes a top-down approach in characterizing the computational challenges of training RecSys with DP-SGD. We use DLRM (deep learning recommendation model [51]) from MLPerf [45] as our target RecSys workload whose default model size is 96 GB. In order to demonstrate the effect embedding table size has on DP-SGD’s training time, we scale down the size of this default model by reducing the number of embedding table entries, from $10 \times \downarrow$ (9.6 GB) to $1000 \times \downarrow$ (96 MB). RecSys training is modeled using PyTorch Opacus [65] which we utilize to evaluate not only the original DP-SGD algorithm (denoted DP-SGD(B)) but also its performance-optimized versions as discussed in Section 2.5,

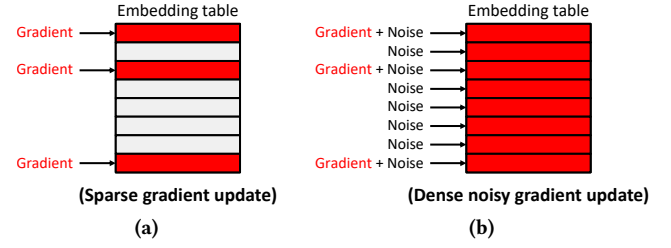


Figure 4. Training an embedding table using (a) SGD and (b) DP-SGD. Example assumes a pooling value of 3. (a) SGD only incurs 3 table reads and 3 table writes during forward propagation and model update, respectively, exhibiting sparse table accesses. (b) In contrast, DP-SGD requires an additional 8 noise write operations on top of SGD’s default 3 table read and 3 table write operations, collectively invoking a dense noisy gradient update during model update.

namely DP-SGD(R) [40] and DP-SGD(F) [13]. Section 6 further details our evaluation methodology.

4.1 Breakdown of End-to-End Training Time

We start our characterization by first trying to gain a high-level understanding on what impact DP training has for RecSys. Figure 3 compares the end-to-end training time of non-private SGD vs. three private DP-SGD designs (DP-SGD(B,R,F)), uncovering several key observations as detailed below.

In general, the training time of SGD remains almost constant regardless of the table size, so we only show a single SGD data point in Figure 3 for brevity. In contrast, all three DP-SGD designs experience an almost linear increase in latency as table size increases. The reason why DP-SGD experiences aggravated training performance is as follows. A non-private SGD training exhibits a *sparse* embedding table access pattern, only touching a small *fixed* number of embeddings (determined by the pooling value and batch size) during both forward propagation (embedding gather) and model update (gradient update), regardless of table size. In contrast, training an embedding table with DP-SGD requires the addition of Gaussian noise to *all* the embeddings within the embedding table during model update, regardless of whether they were targeted for an embedding gather operation during forward propagation. In effect, an SGD’s sparse gradient update turns into a dense *noisy gradient* update with DP-SGD that updates the *entire* embedding table entries, severely pressurizing both compute and memory of the training system (Figure 4).

Consequently, the larger the embedding table size becomes, the more embedding vectors DP-SGD must access to go over the computations required to add noise to the embeddings. An important point to note in Figure 3 is that the performance difference among the three DP-SGD designs gradually diminishes as the model size increases, e.g., the most performance-efficient DP-SGD(F) provides $1.5 \times$

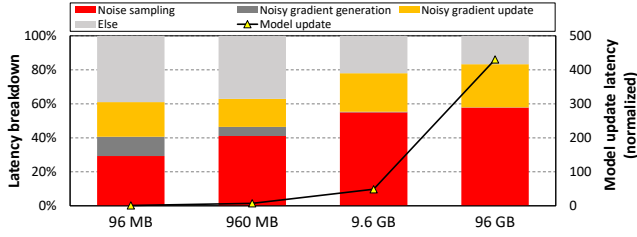


Figure 5. Latency breakdown (left axis) of the model update stage in Figure 3. The right axis shows the latency in model update (normalized to 96 MB), the value of which grows as model size increases.

higher performance than DP-SGD(R) when the table size is the smallest at 96 MB, but the performance gap becomes less than 0.3% under the default size at 96 GB. This is because the performance overhead of updating the embedding table with both noise and gradient (included as part of the *model update* stage in Figure 3) grows proportional to the table size, washing out any performance improvement DP-SGD(R) and DP-SGD(F) provides on top of the baseline DP-SGD(B). In other words, prior work on performance-efficient DP-SGD (DP-SGD(F)), while effective in reducing DP-SGD(B)’s backpropagation latency to derive gradients, is not able to fundamentally address the bottlenecks incurred in training the embedding tables with DP.

Key takeaways: Training embedding tables with DP-SGD requires every single table entry to be added with Gaussian noise, invoking a dense noisy gradient update to the entire table and causing a severe performance overhead.

4.2 Analysis on the Model Update Stage in Private Embedding Table Training

The previous subsection identified the model update stage as becoming a critical performance bottleneck in training private RecSys. In Figure 5, we further break down the latency of the model update stage (red in Figure 3) into four parts: noise sampling, noisy gradient generation, noisy gradient update, and others. We emphasize that, rather than utilizing PyTorch’s built-in functions *as-is* for our characterization, *we heavily optimize and tune the performance of this bottleneck stage to construct a strong, competitive baseline DP-SGD*. More concretely, *our optimized version of the model update stage is 8.2× faster than the baseline implementation using the original built-in PyTorch functions*, reaching 81% of the maximum possible AVX performance achievable for this stage. We further analyze our optimized DP-SGD’s model update stage in the following subsection (Section 4.3).

As depicted in Figure 5, the noise sampling and noisy gradient update account for a significant fraction of the bottlenecked model update stage, with its proportion rapidly increasing as the embedding table size is increased. For instance, the aggregate latency of noise sampling and noisy gradient update accounts for 83.1% of the model update stage

under the default 96 GB model size, consuming 82.8% of its end-to-end training time and becoming the two most significant performance limiters. As previously pointed out in Section 4.1, the amount of memory traffic to *update* the embedding table using the noisy gradient (and accordingly, the overhead of *generating* each noise value to update each embedding vector) grows proportional to the embedding table size, so we can infer that these two bottlenecks will only get worse for future RecSys models with even larger table sizes [46, 67].

Key takeaways: Even with a highly optimized software implementation of DP-SGD, we root-cause its two most significant performance bottlenecks as the step that generates noise and the noisy gradient update step, the overhead of which becomes even worse for future large-scale RecSys.

4.3 Root-causing the Key Challenges behind DP-SGD’s Noise Sampling and Noisy Gradient Update

Since our optimized version of model update stage achieved 8.2× speedup vs. baseline PyTorch’s implementation, a natural question arises: Can the two most bottlenecked stages, i.e., noise sampling and noisy gradient update, be optimized even further and remove their performance bottlenecks completely? Conversely, are noise sampling and noisy gradient update stages fundamental bottlenecks that require alternative measures to address their performance overheads?

To answer these questions, we conducted an in-depth examination of the key primitives that constitute both noise sampling and noisy gradient update. In PyTorch, Gaussian noise sampling function is implemented in the `torch.normal()` API, which generates a tensor of a specified size containing independent random numbers following a normal distribution. To achieve high computational efficiency, this API is implemented based on the Box-Muller algorithm [62], which is well-known to provide high performance random number sampling to processors with vector units. Careful examination of this API (which provides our noise sampling functionality) revealed that its implementation is dominated by conducting a series of trigonometric and logarithmic computations using AVX (Advanced Vector Extensions [32]) instructions. More concretely, we observe that the majority of `torch.normal()`’s execution time is dominated by executing a series of (1) AVX load instruction to retrieve an input vector, (2) 101 AVX compute instructions for trigonometric/logarithmic/other operations, and finally (3) AVX store instruction to write back the result, exhibiting a highly *compute-bound* behavior. The noisy gradient update stage, on the other hand, exhibits a *memory bandwidth limited* behavior as it is primarily a data streaming operation, i.e., each data element is loaded from memory, multiplied by a learning rate, and then added with the existing model weight, requiring only two computations for each loaded data element while streaming a large amount of data.

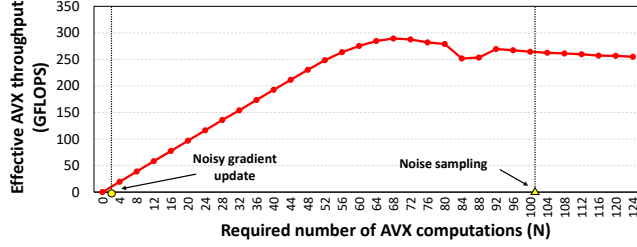


Figure 6. Effective AVX throughput as a function of the number of AVX computations (N) conducted over a single loaded vector. As depicted, our implementation of noise sampling (corresponding to the data point at $N=101$) exhibits a compute-bound behavior and reaches 81% of the maximum AVX performance.

To better illustrate the compute-bound (noise sampling) and memory-bound (noisy gradient update) nature of model update stage, we design a microbenchmark that conducts (1) an AVX vector load from memory, (2) performs N consecutive AVX computations over the loaded vector, and then (3) writes back the resulting vector into memory using AVX store. We then sweep the value of N and measure the effective AVX throughput, the result of which is illustrated in Figure 6. As shown, when the number of N (number of AVX computations performed over the loaded vector) is small, the benchmark exhibits a memory-bound behavior whereas for large N values, it becomes compute-bound. The compute-bound noise sampling stage, with its large N value of 101, achieves an effective AVX throughput of 215 GFLOPS and reaches 81% of the maximum possible AVX performance. As for the memory-bound noisy gradient update, which corresponds to a small N value of 2, it is able to reap out 85.5% of theoretical memory bandwidth and 99.8% of the maximum possible AVX performance. Overall, these results confirm that the noise sampling and noisy gradient update operators we utilize as our baseline DP-SGD(B,R,F) leave little performance left on the table because they already reach close to the optimal performance under the training system’s available compute and memory throughput constraints.

Key takeaways: DP-SGD’s performance limiting noise sampling and noisy gradient update operations already reach close to its optimal performance, suggesting that alternative measures must be devised to address the performance bottlenecks of private RecSys training.

5 LazyDP: An Algorithm-Software Co-Design for Differentially Private RecSys Training

In this section, we present our LazyDP system, an algorithm-software co-design for private RecSys training. LazyDP enables high throughput DP-SGD training while also ensuring that the trained RecSys model brings the same level of privacy protection provided with baseline DP-SGD against the practical adversary discussed in Section 3. Section 5.1 first

introduces the design principles behind LazyDP and the key observations that underpin our proposal. We then detail LazyDP’s lazy noise update algorithm co-designed with our aggregated noise sampling technique for scalable DP-SGD training (Section 5.2). Lastly, Section 5.3 describes LazyDP’s software architecture and its user interface.

5.1 Design Principles and Key Observations

An important observation behind LazyDP is that delaying the noise update process (which we refer to as *lazy noise update*) of DP-SGD can significantly improve computational efficiency while still ensuring that the privacy of final training outcome (i.e., the trained RecSys model) is not affected. We use Figure 7 as a running example to motivate our lazy noise update algorithm, describing key differences between SGD, DP-SGD, and our proposed LazyDP.

In this example, we show the set of gradient and/or noise updates a *single* embedding vector experiences during the course of eight consecutive training iterations. Due to embedding table’s sparse access pattern, the example assumes that an embedding gather operation happens to this particular embedding vector only during the 4th and the 7th training iteration. Recall that SGD only updates the weights of those embedding vectors that are gathered during forward propagation. As such, the non-private SGD updates this embedding vector only during the iterations that it actually accessed, i.e., G_4 and G_7 is derived during the 4th and the 7th iteration to update this embedding vector (Figure 7(a)). The baseline DP-SGD training, on the other hand, adds noise to each embedding vector in every training iteration, significantly amplifying memory read and write traffic compared to SGD (all N_i that update noise to this embedding, Figure 7(b)).

LazyDP’s design is based on the key observation that, as long as we make sure that any delayed noise updates are conducted *before* the actual embedding access occurs, the exact timing of *when* those delayed noise updates were performed have no impact on the gradient values derived ($\tilde{G}_4$ and $\tilde{G}_7$). Consider the embedding vector update that happens on the 4th iteration in Figure 7(c). Here, LazyDP delays the noise updates until the 3rd training iteration (which immediately precedes the iteration that actually accesses the target embedding, i.e., the 4th iteration) and conducts the noise update in aggregation by adding $(N_1+N_2+N_3)$ altogether. This allows gradient $\tilde{G}_4$ to be derived in the 4th iteration using the embedding with a value of $(E+N_1+N_2+N_3)$, the same value that was used to derive $\tilde{G}_4$ under baseline DP-SGD. Unlike DP-SGD, however, the number of memory reads to fetch the embedding vectors (as well as memory writes to store the updated embeddings) is significantly reduced as the three consecutive noise updates of $(N_1+N_2+N_3)$ are performed all at once, rather than going over each noise update separately in each training iteration (Figure 7(b)). Generalizing this behavior across the entire embedding table, we can infer that the number of (delayed) noise updates invoked

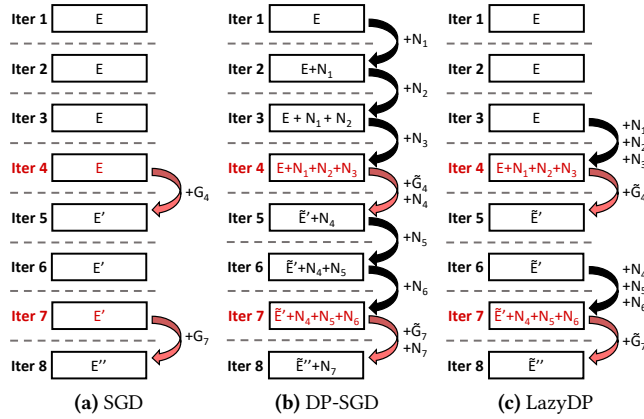


Figure 7. An example embedding's gradient and noise update process using (a) SGD, (b) conventional DP-SGD, and (c) LazyDP's lazy noise update algorithm. The red arrow indicates an embedding update with gradient G_i , a value derived during the i -th training iteration. The black arrow indicates an update with the noise value of N_i , which is the noise that should be updated in iteration i . The tilde symbol represents a value derived with DP in mind (e.g., $\tilde{G}_i$, $\tilde{E}'$ and $\tilde{E}''$ are values derived with noise values considered in its derivation).

with LazyDP is primarily determined by the embedding table pooling value (i.e., number of embedding gathers per table) per each iteration. This is because LazyDP's lazy noise update only occurs for those embeddings that will be gathered during the next training iteration, the number of which is solely determined by the table's pooling value but not the table size. Such property helps LazyDP significantly reduce memory traffic than DP-SGD, which must always update the entire table in every training iteration. Consequently, LazyDP's reduction in memory read/write traffic helps address the bottleneck in the memory-bound noisy gradient update operation (Figure 5), improving training throughput. Nonetheless, it is important to point out that the overhead of *noise sampling* itself (e.g., generating N_1 , N_2 , and N_3) has not changed between Figure 7(b) and Figure 7(c). We discuss how the bottlenecks of noise sampling are addressed in Section 5.2.2 using our aggregated noise sampling algorithm.

Note that, in order to perform the lazy noise updates right before an embedding vector is about to be accessed, LazyDP must be able to *predict* which embedding vectors will be accessed in the subsequent training iteration. Another key observation we make is that the nature of RecSys training dataset enables us to *precisely* determine which embedding vectors will be gathered at what time, which LazyDP can utilize to determine when to initiate lazy noise update. In RecSys, the training dataset describes which embedding table entries to gather from during forward propagation, not just for the current iteration but also for all future iterations as well. LazyDP is designed to prefetch the next training

iteration's training mini-batch in order to analyze which embedding vectors will be accessed in the subsequent iteration and determine whether to initiate lazy noise update or not. Since LazyDP requires visibility on which embeddings will be accessed in one subsequent training iteration, prefetching a single mini-batch in advance is sufficient to implement our proposal. In the following subsection, we detail the implementation of our lazy noise update and aggregated noise sampling algorithm.

5.2 Implementation

5.2.1 Lazy Noise Update Algorithm. In Algorithm 1, we provide a pseudo-code of LazyDP's lazy noise update algorithm. For brevity, the pseudo-code omits the model update process for the dense MLP layers because both DP-SGD(F) and LazyDP apply the identical DP protection for MLP layers. Below we walk through this pseudo-code to describe its working mechanism and pinpoint to where LazyDP's performance improvement comes from.

Line 1-2 introduces the *HistoryTable*, a data structure that tracks the number of delayed noise updates for each embedding vector. A naive implementation of the *HistoryTable* will be to simply count the number of delayed noise updates each embedding vector is pending, incrementing the per-embedding counter value whenever that embedding is not accessed in each training iteration. Considering the significant access sparsity of embedding tables, however, such naive implementation will lead to significant memory write traffic to update the *HistoryTable*. LazyDP instead employs the *HistoryTable* to track the most recent training iteration ID number that performed a lazy noise update to each embedding vector. By calculating the distance between the current training iteration and the ID value stored within the *HistoryTable* (line 14), we can derive the number of delayed noise updates. As the value of *HistoryTable* is only updated for the *sparsely* accessed embedding table entries (line 15), the performance overhead of maintaining and updating the *HistoryTable* is very low.

Line 3-5 introduces the *InputQueue*, which enables LazyDP to preview the next input mini-batch. For the purpose of identifying which embeddings will require lazy noise updates, a queue with a size of two is sufficient, storing two consecutive input mini-batches. When the training system is bootstrapped, the first training mini-batch is retrieved from the data storage system (line 5). During the main training iteration loop (line 6-27), LazyDP fetches the next mini-batch and reuses the previous iteration's next mini-batch as the current mini-batch, having visibility into *both* current and next input mini-batches while only fetching a *single* mini-batch each iteration (line 7) – identical to baseline SGD and DP-SGD.

Line 6-27 explains the key operations undertaken during the main training loop. As described in line 8-10, LazyDP requires no changes to forward and backpropagation, only

Algorithm 1 Lazy Noise Update Algorithm.

(Parameters) model weight: θ , max clipping factor: C , noise multiplier: σ , number of iterations: N , training batch size: B , learning rate: η , embedding dimension: dim , number of embedding vectors in the embedding table: E

```

1:  $\triangleright$  Data structure recording the latest noise updated iter. for each embedding vector
2:  $HistoryTable \leftarrow ZEROS(E)$   $\triangleright$  Initialize to an array of length  $E$  containing zeros
3:  $\triangleright$  Data structure to store the current and next input mini-batches
4:  $InputQueue \leftarrow QUEUE(size = 2)$   $\triangleright$  Contains two consecutive input mini-batches
5:  $InputQueue.push(GETNEXTMINIBATCH)$   $\triangleright$  Load the first training mini-batch
6: for  $iter \leftarrow 1$  to  $N$  do
7:    $InputQueue.push(GETNEXTMINIBATCH)$   $\triangleright$  Load a new training mini-batch
8:    $\triangleright$  Forward and backward propagation is done identically to standard DP-SGD
9:    $loss \leftarrow FORWARDPROPAGATION(InputQueue.head(), \theta)$ 
10:   $gradient \leftarrow BACKPROPAGATION(loss, \theta, C)$   $\triangleright$  Training with a grad clipping
11:   $\triangleright$  Retrieve the next iteration's input mini-batch from the tail of the InputQueue
12:   $next\_accesses \leftarrow InputQueue.tail().sparse\_inputs$ 
13:  for each  $idx$  in  $next\_accesses$  do  $\triangleright$  Derive number of delayed noise updates
14:     $delays[idx] \leftarrow iter - HistoryTable[idx]$ 
15:     $HistoryTable[idx] \leftarrow iter$   $\triangleright$  Renew the value of the HistoryTable
16:  end for
17:   $\triangleright$  Noise sampling using next mini-batch and number of delayed noise updates
18:   $noise \leftarrow NOISESAMPLING(next\_accesses, delays, B, C, dim)$ 
19:   $\triangleright$  Generate noisy gradient
20:   $noisy\_gradient \leftarrow gradient + noise$   $\triangleright$  Merge sparse gradient and sparse noise
21:   $\triangleright$  Model update
22:  for each  $idx$  in  $noisy\_gradient.indices$  do
23:     $\triangleright$  Weight update
24:     $\theta_{sparse}[idx] \leftarrow \theta_{sparse}[idx] - \eta \times noisy\_gradient[idx]$ 
25:  end for
26:   $InputQueue.pop()$   $\triangleright$  Pop the head of the InputQueue
27: end for

28: procedure NOISESAMPLING( $next\_accesses, delays, B, C, dim$ )
29:   for each  $idx$  in  $next\_accesses$  do
30:     if ANS disabled then
31:        $\triangleright$  Accumulate Gaussian noises of size  $dim$ , variance of  $\sigma^2 C^2$  iteratively
32:        $noise[idx] \leftarrow ZEROS(dim) + noise[i]$ ; noise vector for  $i^{th}$  embedding
33:       for  $n \leftarrow 1$  to  $delays[idx]$  do
34:          $noise[idx] \leftarrow noise[idx] + \frac{1}{B} \times GAUSSIANNOISE(\sigma^2 C^2, dim)$ 
35:       end for
36:     else  $\triangleright$  ANS enabled
37:        $\triangleright$  Generate an aggregated noise all at once using the ANS algorithm
38:        $noise[idx] \leftarrow \frac{1}{B} \times GAUSSIANNOISE(delays[idx] \times \sigma^2 C^2, dim)$ 
39:     end if
40:   end for
41:   return noise
42: end procedure

```

changing the way the gradient and noise are used to update the embedding table. Line 11-27 describes the important modifications required for lazy noise update, which performs noise sampling, noisy gradient generation, and noisy gradient update for model training. Because lazy noise update targets embeddings that will be accessed in the next iteration, the tail of the *InputQueue* is used (line 12) to identify the embedding vectors that will lazily be updated with noise. We then use the *HistoryTable* to calculate the number of delayed noise updates to perform for the target embedding vectors (line 13-16). The next process is the noise sampling stage (line 18) which samples the delayed amount of noise to add and then accumulate to generate the final noise vector. Depending on whether LazyDP's aggregated noise sampling is employed or not (line 28-40), noise sampling can incur significantly different computational overheads, the details of which are discussed in the Section 5.2.2.

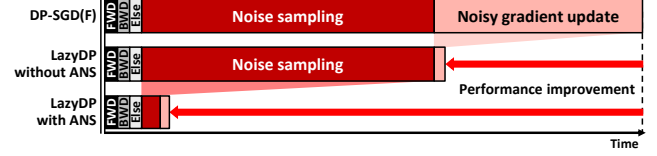


Figure 8. LazyDP's effect on latency reduction.

After noise sampling, LazyDP generates the noisy gradient by merging the sampled noise with the gradient (line 19-20). The noisy gradient is then utilized to update the model based on the optimizer function (line 22-25). The performance gain of the lazy noise update comes from this phase. In baseline DP-SGD, the size of the noisy gradient is equivalent to the size of the entire embedding table so updating the model exhibits a memory bandwidth limited behavior. With our LazyDP, however, the size of the noisy gradient is identical to the size of the embedding vectors accessed in the current and next iterations (line 20). Given the sparse access pattern of embedding tables, LazyDP's noisy gradient becomes orders of magnitude smaller in size than DP-SGD's dense noisy gradient, providing significant reduction in the memory traffic for model update.

5.2.2 Aggregated Noise Sampling Algorithm. Although lazy noise update alleviates the memory bottlenecks of noisy gradient update, the computational challenge of noise sampling still remains (Figure 8). This is because the total number of Gaussian noise to generate and accumulate into accessed embeddings has not changed vs. baseline DP-SGD, rendering noise sampling to still cause a performance bottleneck. We present our *aggregated noise sampling* (ANS) algorithm, which leverages the mathematical property of normally distributed random variables to substantially reduce the number of noise sampling. In probability theory, it is well known the summation of two or more random variables sampled from a normal distribution follows another normal distribution, but one with a different mean and variance [18]. The following theorem describes this mathematical property.

Theorem 5.1 (Sum of i.i.d. normal random variables [18]). Let $X_1, X_2, \dots, X_n$ be independent and identically distributed (i.i.d.) Gaussian random variables with a mean value of 0 and a variance of σ^2 . Then, the summation of these random variables, denoted by Y , follows a Gaussian distribution with a mean of 0 and a variance of $n\sigma^2$. Specifically, we have:

$$X_i \sim N(0, \sigma^2) \quad \text{for } i = 1, 2, \dots, n,$$

where all X_i are independent variables following Gaussian distribution with a mean of 0 and a variance of σ^2 . The summation of these independent variables is given by:

$$Y = X_1 + X_2 + \dots + X_n, \quad Y \sim N(0, n\sigma^2).$$

In effect, what this theorem implies is that we can replace the summation of multiple independently sampled Gaussian noise into a *single* Gaussian noise sampled from

a normal distribution with a larger variance. Coming back to our lazy noise update algorithm in Algorithm 1, without ANS (line 30), the compute-bound noise sampling operation must be invoked for each delayed noise update (e.g., 3 separate noise sampling operations to derive N_1 , N_2 , and N_3 in Figure 7(c)), causing significant computation overheads. However, with the ANS algorithm, we can substitute multiple noise sampling operations (line 33-35) with a just a single *GaussianNoise* sampling (line 38) from a normal distribution with a variance of $(delay \times \sigma^2 C^2)$, instead of $\sigma^2 C^2$, significantly reducing the overhead of noise sampling (e.g., a single noise sampling operation derives $N_1+N_2+N_3$ in Figure 7(c)).

5.3 Software Architecture

User interface. LazyDP is designed as a software plug-in to PyTorch Opacus [65], extending the capabilities of existing ML frameworks to seamlessly support scalable DP-SGD training for RecSys. Figure 9(a) shows the user interface provided with LazyDP, which transforms existing PyTorch model, optimizer, and data_loader instances into LazyDP-enabled instances. The wrapper API is also designed to accept various training hyperparameters to explore models with different model accuracy, privacy budget, and training performance.

Software system. The key components of LazyDP are implemented as part of the backend software runtime system of PyTorch Opacus. Figure 9(b) provides a high-level overview of the LazyDP software architecture, which wraps around PyTorch’s data_loader, model, and optimizer instances to implement our proposal. We primarily utilize the DP-enabled data loader and model instances from PyTorch Opacus as-is to provide the same key features required for DP-SGD training, e.g., Poisson sampling based training dataset retrieval (data loader instance) and per-example gradient derivation (model instance). To provide visibility into both the current and the next training iteration’s mini-batch inputs, however, LazyDP’s data loader instance is augmented with a two-entry input queue, which is forwarded to LazyDP’s model and optimizer instances for DP-based gradient derivation and model update, respectively. The LazyDP optimizer instance incorporates our key optimizations, the *ANS engine* and the *noise update engine*, which implements the aggregated noise sampling and the lazy noise update algorithm, respectively.

Putting everything together, the synergistic combination of the noise update engine and the ANS engine elegantly addresses the two most severe performance bottlenecks of private RecSys training, the compute-bound noise sampling and the memory-bound noisy gradient update. It is important to note that *ANS builds on top of LazyDP’s ability to postpone noise sampling and noisy gradient update operations, a feature enabled by our lazy noise update algorithm*, highlighting the importance of LazyDP’s algorithm-software co-design.

```
# Instantiate model, optimizer, and data_loader using PyTorch
model = Net()
optimizer = SGD(model.parameters(), lr=0.05)
data_loader = torch.utils.data.DataLoader(dataset, batch_size=1024)

# Apply LazyDP wrapper to the model, optimizer, and data_loader
model, optimizer, data_loader = LazyDP.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=data_loader,
    noise_multiplier=1.1,
    max_gradient_norm=1.0
)
```

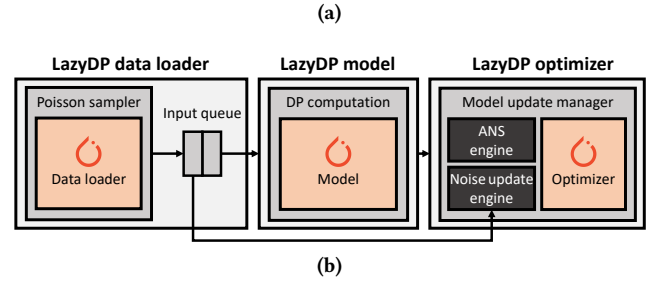


Figure 9. LazyDP’s (a) user interface and (b) software architecture.

6 Methodology

System configuration. All our evaluations are conducted on a server containing NVIDIA V100 GPU (32 GB HBM2, 900 GB/sec bandwidth) and Intel Xeon E5-2698v4 CPU (256 GB of DDR4 DRAM, 68 GB/sec bandwidth) connected with PCIe 3.0 (16 GB/sec bandwidth). We assume a hybrid CPU-GPU system (Section 2.2) focusing on large-scale RecSys with several hundreds of GBs of embedding tables [37, 38, 64, 67], where the CPU is used to train the sparse embedding layers and the GPU for the dense DNN layers.

Software. We use PyTorch (v.1.12.1) [56] and its Opacus [65] extensions for modeling the key functionalities required for DP-SGD, e.g., per-example gradient derivation, gradient clipping, Gaussian noise sampling, etc. However, as we detailed in Section 4.2, the built-in functions that come with PyTorch Opacus as-is exhibited sub-optimal performance. For a conservative evaluation of LazyDP, we heavily optimize the performance of the key bottleneck layers in baseline DP-SGD by using Intel TBB [30] and OpenMP [5] to optimally tune the way it leverages data-level parallelism (vectorized execution) and thread-level parallelism (multi-threading), achieving 13.4× higher performance than the built-in PyTorch implementations and reaching 81% of the ideal performance achievable (see Section 4.3). This performance optimized version of DP-SGD is used in all of our evaluations reported in this paper (the characterization study in Section 4 and the evaluation section in Section 7). Note that, because DP-SGD(F) [13] consistently exhibited higher performance than DP-SGD(R) [40], we only show the results using DP-SGD(F) when comparing against LazyDP in Section 7 for brevity.

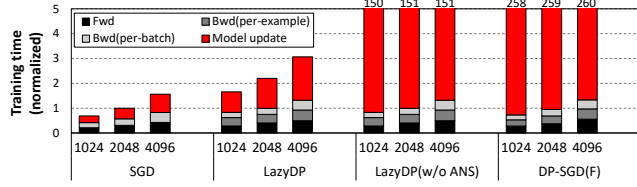


Figure 10. End-to-end training time, normalized to SGD trained with mini-batch size 2,048.

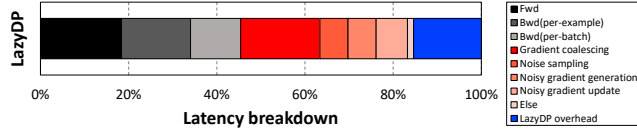


Figure 11. Breakdown of LazyDP's training time when trained with mini-batch size 2,048.

Benchmarks. We use the open-sourced DLRM [51] and follow the MLPerf (v2.1) [45] recommendation training benchmark's configuration to model our default RecSys model architecture, which uses 8 MLP layers and 26 embedding layers with a 128-dimensional embedding, resulting in a total model size of 96 GB. The embedding table access patterns are drawn from a uniform distribution. In Section 7.3, we study the sensitivity of LazyDP when deviating from this default configuration.

7 Evaluation

In this section, we first evaluate the performance and energy-efficiency of LazyDP (Section 7.1). We then discuss LazyDP's implementation overhead (Section 7.2) and its sensitivity to different RecSys configurations (Section 7.3). Finally, LazyDP is compared against EANA [52], a prior work that provides high-performance private RecSys training at the cost of reduced privacy guarantee (Section 7.4).

7.1 Performance and Energy-Efficiency

Performance. In Figure 10, we compare the training time of non-private SGD and the private DP-SGD(F) and LazyDP over three different training mini-batch sizes (1,024, 2,048, and 4,096). To demonstrate the effectiveness of our ANS optimization, we show both LazyDP *without* ANS (denoted LazyDP(w/o ANS)) and LazyDP with all of its optimizations incorporated (denoted LazyDP). As discussed in Section 4, training with DP-SGD(F) incurs significant latency overheads during the noise sampling and noise gradient update process, causing a 166 – 375× performance slowdown vs. non-private SGD. LazyDP(w/o ANS) helps alleviate the bottleneck incurred in noisy gradient update, so it provides an average 72% speedup on top of DP-SGD(F). However, the performance overhead of noise sampling still remains with LazyDP(w/o ANS) which renders its performance to still fall behind SGD by 97 – 218×. LazyDP with ANS holistically

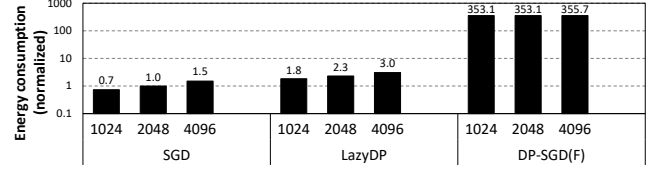


Figure 12. Energy consumption (normalized to SGD, batch: 2,048).

addresses all the critical bottlenecks of DP-SGD(F) and provides 85 – 155× speedup, bringing down private DP-SGD's training time to be on par with the non-private SGD. Specifically, our LazyDP only incurs 1.96× to 2.42× slowdown over the non-private SGD, enabling private RecSys training to become practical for ML practitioners.

Latency breakdown. To better illustrate where LazyDP's speedup comes from, we show in Figure 11 a latency breakdown of LazyDP under the default configuration trained with mini-batch size 2,048. Here, we further divide LazyDP's model update stage (red bar in Figure 10) into finer granularity where the conventional operations conducted with DP-SGD training (e.g., noise sampling, noisy gradient generation, ...) are colored in reds and the pure LazyDP-introduced latency overhead is colored in blue. Because LazyDP's lazy noise update algorithm (Algorithm 1) and ANS (Section 5.2.2) effectively reduce the latency of noise sampling and noisy gradient update operations by 1,081× and 418×, respectively, no single operation is left as a major bottleneck anymore. The latency overheads of LazyDP (the blue bar) include operations such as updating the *HistoryTable*, identifying which embeddings to initiate lazy noise updates, and others, but all of these only account for 15% of the training time of LazyDP. Specifically, this 15% latency overhead of LazyDP is further broken down into three distinct components: (1) removing duplicated embedding indices among the embeddings accessed next, (2) reading the *HistoryTable* and calculating the standard deviation for ANS, and (3) updating the *HistoryTable*, each component accounting for 61%, 22%, and 17% to LazyDP's overhead, respectively.

Energy-efficiency. LazyDP is a software-based solution that works seamlessly on top of existing systems. Consequently, the reduction in training time directly translates into improved energy-efficiency. We utilize Intel's *pcm-power* [31] and NVIDIA's *nvidia-smi* [53] to measure the power consumption of CPU and GPU, respectively, multiplying the aggregated CPU-GPU power number with its corresponding training time to calculate each design's energy consumption. As shown in Figure 12, LazyDP provides significant improvements in energy-efficiency by reducing energy consumption by an average 155× against DP-SGD(F).

7.2 Implementation Overhead

As discussed in Section 5.2, our LazyDP requires additional metadata such as (1) the delayed input data queue that provides visibility into the next training iteration's embedding

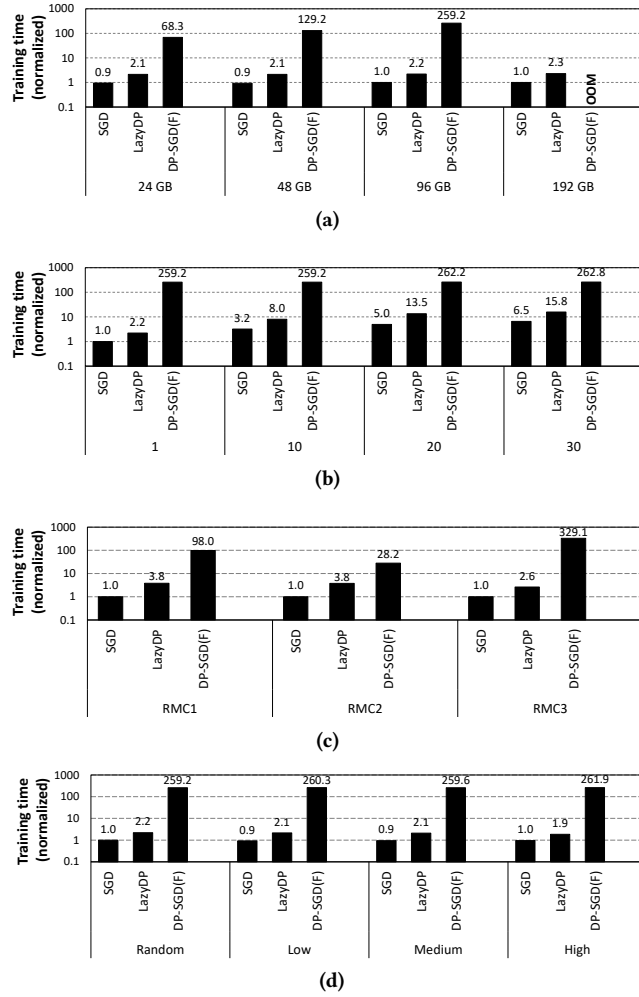


Figure 13. LazyDP sensitivity to (a) embedding table size (24/48/96/192 GB), (b) number of embedding gathers per table, i.e., the embedding pooling factor (1/10/20/30), (c) alternative DLRM model configurations, and (d) alternative training dataset.

table accesses, and (2) the *HistoryTable* that tracks the number of delayed noise updates. As the input data queue only requires storing one additional training mini-batch, it incurs 213 KB (mini-batch size $\times$ number of embedding tables $\times$ average lookups per embedding table $\times$ 4 Bytes) of additional storage overhead. As for the *HistoryTable*, for our default 96 GB model configuration, it consumes an additional 751 MB (total number of embedding vectors $\times$ 4 Bytes) of memory, which is less than 1% of the total model size.

7.3 Sensitivity

Embedding table size. Figure 13(a) compares the training time of LazyDP against SGD and DP-SGD(F) when the embedding table size is changed. Regardless of the table size, the latency of SGD and LazyDP remains almost constant. This is because the performance of SGD and LazyDP is mostly

determined as a function of the number of embedding vectors gathered from the table, irrespective of the table size. The training time of DP-SGD(F), on the other hand, scales proportional to the embedding table size, causing an out-of-memory (OOM) error when the table size reaches 192 GB. These results highlight the high scalability of LazyDP, demonstrating its applicability for future large-scale RecSys models with even larger embedding table sizes.

Embedding pooling value. Figure 13(b) summarizes the effect of the number of embedding vector gathers (i.e., embedding pooling value) on training time (normalized to the leftmost SGD design). Due to the large overhead incurred with its noisy model updates, the latency of DP-SGD(F) is already significantly high that the additional overhead caused by having a larger embedding pooling value becomes marginal beyond pooling value of 10. As for SGD and LazyDP, the effect of noisy model update on performance is non-existent (SGD) or close to zero (LazyDP). This means that a larger pooling value causes a higher memory bandwidth pressure and accordingly, a higher latency overhead for SGD and LazyDP, which explains its gradual increase in training time with larger pooling values. Consequently, the performance gap between LazyDP and DP-SGD(F) becomes narrower as the pooling value is increased. Nonetheless, LazyDP still provides significant benefits and achieves 16.7 $\times$ speedup vs. DP-SGD(F) with pooling value of 30.

Alternative DLRM model configurations. To further demonstrate the robustness of LazyDP, we examine the effect of changing both embedding table size and embedding pooling value by studying different DLRM model configurations discussed in [26, 27]. As depicted in Figure 13(c), LazyDP consistently provides a significant 52.7 $\times$ average speedup compared to DP-SGD(F). With this new model architecture, LazyDP incurs less than 3.1% memory capacity overhead across all studied models. As for the performance overhead, LazyDP introduces latency to identify which embeddings to apply noisy updates and etc, which accounts for 8.9% to 11.9% of LazyDP's end-to-end training time.

Alternative training dataset. We also study the robustness of LazyDP to alternative training dataset by using the Kaggle DAC dataset [33]. Specifically, we follow the methodology from [38] and construct three distinct datasets where each exhibits low, medium, and high embedding table access skewness, i.e., 90% of the embedding table accesses are concentrated on 36%, 10%, and 0.6% of table entries for our low, medium, and high skewness dataset. As illustrated in Figure 13(d), DP-SGD(F) exhibits minimal variation in its training time because its end-to-end training time is completely bottlenecked by the model update stage, suffering from significant slowdown irrespective of table access locality. LazyDP again shows robustness and achieves an average 129.03 $\times$ speedup vs. DP-SGD(F). There is no additional memory capacity overhead with this new training dataset as it is primarily determined by the model configuration. In terms

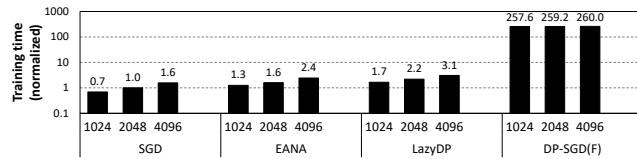


Figure 14. Training time of LazyDP vs. EANA over different training mini-batch sizes. All bars are normalized to SGD trained with mini-batch size 2,048.

of performance overhead, the additional latency introduced with LazyDP is shown to be always below 14% of LazyDP’s end-to-end training time, similar to the default training configuration.

7.4 LazyDP vs. EANA

EANA [52] is a recently proposed private training algorithm for RecSys, presenting an alternative DP-SGD algorithm for higher performance. EANA alleviates the performance overhead of private embedding table training by adding noise *only* to the accessed embedding vectors in each training iteration. While such modification helps improve training throughput, the privacy of EANA is highly *data-dependent* and becomes weaker when the embedding table access pattern exhibits a skewed access distribution [52]. Prior work [34, 35, 38, 41, 64] reports that the embedding table access pattern in RecSys exhibits a power-law distribution (i.e., a small number of hot embedding vectors receive very high access frequency while the other cold vectors receive a small number of accesses), and EANA becomes less useful for such real world RecSys models. Overall, our LazyDP is designed to satisfy the dual requirements of high performance and privacy guarantee from the ground up. We implement EANA and compare against LazyDP in Figure 14. LazyDP only incurs 27% to 37% performance overhead over EANA while guaranteeing the same level of privacy provided with the vanilla DP-SGD algorithm.

8 Conclusion

This paper proposes LazyDP, our algorithm-software co-design that provides scalable and high-performance private RecSys training. We first characterize real world RecSys models and uncover several critical performance bottlenecks such as the compute-limited noise sampling and memory-limited noisy gradient update operations. LazyDP successfully addresses these two bottlenecks via our lazy noise update and aggregated noise sampling optimizations, providing substantial performance speedup while ensuring mathematically equivalent, differentially private RecSys models to be trained.

Acknowledgment

This research is supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government

(MSIT) (NRF-2021R1A2C2091753), Samsung Advanced Institute of Technology of Samsung Electronics Co., Ltd, and Samsung Electronics Co., Ltd (IO201210-07974-01). Minsoo Rhu is partially supported by the Google Research Scholar Award.

References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep Learning with Differential Privacy. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [2] Rohan Anil, Badih Ghazi, Vineet Gupta, Ravi Kumar, and Pasin Manurangsi. Large-Scale Differentially Private BERT. In *arxiv.org*, 2021.
- [3] Apple. Learning with Privacy at Scale. <https://docs-assets.developer.apple.com/ml-research/papers/learning-with-privacy-at-scale.pdf>, 2017.
- [4] Moumita Bhattacharya and Sudarshan Lamkhede. Augmenting Netflix Search with In-Session Adapted Recommendations. In *Proceedings of the ACM Conference on Recommender Systems (RECSYS)*, 2022.
- [5] OpenMP Architecture Review Board. OpenMP 4.5 API C/C++ Syntax Reference Guide. <https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf>, 2015.
- [6] Nicholas Carlini. Privacy Considerations in Large Language Models. <https://ai.googleblog.com/2020/12/privacy-considerations-in-large.html>, 2020.
- [7] Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwal, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting Training Data from Diffusion Models. In *arxiv.org*, 2023.
- [8] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom B. Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting Training Data from Large Language Models. In *Proceedings of the USENIX Security Symposium*, 2021.
- [9] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, 2016.
- [10] Christopher A. Choquette-Choo, Florian Tramèr, Nicholas Carlini, and Nicolas Papernot. Label-Only Membership Inference Attacks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.
- [11] Paul Covington, Jay Adams, and Emre Sargin. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the ACM Conference on Recommender Systems (RECSYS)*, 2016.
- [12] Soham De, Leonard Berrada, Jamie Hayes, Samuel L. Smith, and Borja Balle. Unlocking High-Accuracy Differentially Private Image Classification through Scale. In *arxiv.org*, 2022.
- [13] Carson Denison, Badih Ghazi, Pritish Kamath, Ravi Kumar, Pasin Manurangsi, Krishna Giri Narra, Amer Sinha, Avinash Varadarajan, and Chiyuan Zhang. Private Ad Modeling with DP-SGD. In *arxiv.org*, 2022.
- [14] Tom Diethe, Oluwaseyi Feyisetan, Borja Balle, and Thomas Drake. Preserving Privacy in Analyses of Textual Data. In *Proceedings of the International Conference on Web Search and Data Mining*, 2020.
- [15] Cynthia Dwork. Differential Privacy. In *Automata, Languages and Programming*, 2006.
- [16] Cynthia Dwork. Differential Privacy: A Survey of Results. In *Theory and Applications of Models of Computation*, 2008.
- [17] Cynthia Dwork and Aaron Roth. The Algorithmic Foundations of Differential Privacy. In *Found. Trends Theor. Comput. Sci.*, 2014.

- [18] Bennett Eisenberg and Rosemary Sullivan. Why Is the Sum of Independent Normal Random Variables Normal? In *Mathematics Magazine*, 2008.
- [19] Ali Mamdouh Elkahky, Yang Song, and Xiaodong He. A Multi-View Deep Learning Approach for Cross Domain User Modeling in Recommendation Systems. In *Proceedings of the International Conference on World Wide Web (WWW)*, 2015.
- [20] Facebook. Accelerating Facebook's Infrastructure with Application-specific Hardware. <https://code.fb.com/data-center-engineering/accelerating-infrastructure/>, 2019.
- [21] Vitaly Feldman, Ilya Mironov, Kunal Talwar, and Abhradeep Thakurta. Privacy Amplification by Iteration. In *IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, 2018.
- [22] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015.
- [23] Matthew Fredrikson, Eric Lantz, Somesh Jha, Simon M. Lin, David Page, and Thomas Ristenpart. Privacy in Pharmacogenetics: An End-to-End Case Study of Personalized Warfarin Dosing. In *Proceedings of the USENIX Security Symposium*, 2014.
- [24] Amin Ghiasi, Hamid Kazemi, Steven Reich, Chen Zhu, Micah Goldblum, and Tom Goldstein. Plug-In Inversion: Model-Agnostic Inversion for Vision with Data Augmentations. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.
- [25] Aditya Golatkar, Alessandro Achille, Yu-Xiang Wang, Aaron Roth, Michael Kearns, and Stefano Soatto. Mixed Differential Privacy in Computer Vision. In *Proceedings of the Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [26] Udit Gupta, Samuel Hsia, Vikram Saraph, Xiaodong Wang, Brandon Reagen, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. DeepRecSys: A System for Optimizing End-To-End At-Scale Neural Recommendation Inference. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [27] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottle, Kim Hazelwood, Mark Hempstead, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, and Xuan Zhang. The Architectural Implications of Facebook's DNN-Based Personalized Recommendation. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.
- [28] Niv Haim, Gal Vardi, Gilad Yehudai, Ohad Shamir, and Michal Irani. Reconstructing Training Data From Trained Neural Networks. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*, 2022.
- [29] Shlomo Hoory, Amir Feder, Avichai Tendler, Sofia Erell, Alon Cohen, Itay Laish, Hootan Nakhost, Uri Stemmer, Ayelet Benjamini, Avinatan Hassidim, and Yossi Matias. Learning and Evaluating a Differentially Private Pre-trained Language Model. In *Findings of the Association for Computational Linguistics: EMNLP*, 2021.
- [30] Intel. Intel oneAPI Threading Building Blocks. <https://software.intel.com/en-us/tbb>, 2023.
- [31] Intel. Intel Performance Counter Monitor (Intel PCM). <https://github.com/intel/pcm>, 2023.
- [32] Intel. Intrinsics for Intel Advanced Vector Extensions 2 (Intel AVX2). <https://www.intel.com/content/www/us/en/docs/cpp-compiler/developer-guide-reference/2021-10/intrinsics-for-avx2.html>, 2023.
- [33] Kaggle. Criteo Display Advertising Challenge. <https://www.kaggle.com/c/criteo-display-ad-challenge>, 2014.
- [34] Hongju Kal, Seokmin Lee, Gun Ko, and Won Woo Ro. SPACE: Locality-aware Processing in Heterogeneous Memory for Personalized Recommendations. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2021.
- [35] Liu Ke, Udit Gupta, Carole-Jean Wu, Benjamin Youngjae Cho, Mark Hempstead, Brandon Reagen, Xuan Zhang, David Brooks, Vikas Chandra, Utku Diril, et al. RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [36] Alexey Kurakin, Shuang Song, Steve Chien, Roxana Geambasu, Andreas Terzis, and Abhradeep Thakurta. Toward Training at ImageNet Scale with Differential Privacy. In *arxiv.org*, 2022.
- [37] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. Tensor Casting: Co-Designing Algorithm-Architecture for Personalized Recommendation Training. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.
- [38] Youngeun Kwon and Minsoo Rhu. Training Personalized Recommendation Systems from (GPU) Scratch: Look Forward Not Backwards. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2022.
- [39] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-Based Learning Applied to Document Recognition. In *Proceedings of the IEEE*, 1998.
- [40] Jaewoo Lee and Daniel Kifer. Scaling up Differentially Private Deep Learning with Fast Per-Example Gradient Clipping. In *Proceedings on Privacy Enhancing Technologies (PoPET)*, 2021.
- [41] Yejin Lee, Seong Hoon Seo, Hyunji Choi, Hyoung Uk Sul, Soosung Kim, Jae W Lee, and Tae Jun Ham. MERCI: Efficient Embedding Reduction on Commodity Hardware Via Sub-query Memoization. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2021.
- [42] Xuechen Li, Florian Tramer, Percy Liang, and Tatsunori Hashimoto. Large Language Models Can Be Strong Differentially Private Learners. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [43] Liang Luo, Jacob Nelson, Luis Ceze, Amar Phanishayee, and Arvind Krishnamurthy. Parameter Hub: a Rack-Scale Parameter Server for Distributed Deep Neural Network Training. In *ACM Symposium on Cloud Computing (SoCC)*, 2018.
- [44] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [45] MLCommons. MLPerf v2.1 Training Benchmarks. <https://mlcommons.org/en/training-normal-21/>, 2022.
- [46] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie (Amy) Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, KR Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. Software-Hardware Co-Design for Fast and Scalable Training of Deep Learning Recommendation Models. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2022.
- [47] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, et al. High-performance, Distributed Training of Large-scale Deep Learning Recommendation Models. In *arxiv.org*, 2021.
- [48] Milad Nasr, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Eric Wallace, Florian Tramèr, and Katherine Lee. Scalable Extraction of Training Data from (Production) Language Models. In *arxiv.org*,

- 2023.
- [49] Milad Nasr, Reza Shokri, and Amir Houmansadr. Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning. In *IEEE Symposium on Security and Privacy (SP)*, 2019.
 - [50] Milad Nasr, Shuang Song, Abhradeep Thakurta, Nicolas Papernot, and Nicholas Carlini. Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning. In *IEEE Symposium on Security and Privacy (SP)*, 2021.
 - [51] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep Learning Recommendation Model for Personalization and Recommendation Systems. In *arxiv.org*, 2019.
 - [52] Lin Ning, Steve Chien, Shuang Song, Mei Chen, Yunqi Xue, and Devora Berlowitz. EANA: Reducing Privacy Risk on Large-Scale Recommendation Models. In *Proceedings of the ACM Conference on Recommender Systems (RECSYS)*, 2022.
 - [53] NVIDIA. System Management Interface SMI. <https://developer.nvidia.com/nvidia-system-management-interface>, 2016.
 - [54] Beomsik Park, Ranggi Hwang, Dongho Yoon, Yoonhyuk Choi, and Minsoo Rhu. DiVa: An Accelerator for Differentially Private Machine Learning. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2022.
 - [55] Natalia Ponomareva, Jasmijn Bastings, and Sergei Vassilvitskii. Training Text-to-Text Transformers with Privacy Guarantees. In *Proceedings of the Fourth Workshop on Privacy in Natural Language Processing*, 2022.
 - [56] PyTorch. <http://pytorch.org>, 2022.
 - [57] Rachit Rajat, Yongqin Wang, and Murali Annavaram. LAORAM: A Look Ahead ORAM Architecture for Training Large Embedding Tables. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2023.
 - [58] Amedeo Sapio, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtárik. Scaling Distributed Machine Learning with In-Network Aggregation. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2021.
 - [59] Branislav Stojkovic, Jonathan Woodbridge, Zhihan Fang, Jerry Cai, Andrey Petrov, Sathya Iyer, Daoyu Huang, Patrick Yau, Arvind Kumar, Hitesh Jawa, and Anamita Guha. Applied Federated Learning: Architectural Design for Robust and Efficient Learning in Privacy Aware Settings. In *arxiv.org*, 2022.
 - [60] Lukas Struppek, Dominik Hintersdorf, Antonio De Almeida Correia, Antonia Adler, and Kristian Kersting. Plug & Play Attacks: Towards Robust and Flexible Model Inversion Attacks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.
 - [61] Lauren Watson, Chuan Guo, Graham Cormode, and Alexandre Sablayrolles. On the Importance of Difficulty Calibration in Membership Inference Attacks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
 - [62] Wikipedia. Box-Muller Transform. https://en.wikipedia.org/wiki/Box-Muller_transform, 2023.
 - [63] Xinyang Yi, Yi-Fan Chen, Sukriti Ramesh, Vinu Rajashekhar, Lichan Hong, Noah Fiedel, Nandini Seshadri, Lukasz Heldt, Xiang Wu, and EH Chi. Factorized Deep Retrieval and Distributed TensorFlow Serving. In *Proceedings of Conference on Machine Learning and Systems (MLSys)*, 2018.
 - [64] Chunxing Yin, Bilge Acun, Carole-Jean Wu, and Xing Liu. TT-Rec: Tensor Train Compression for Deep Learning Recommendation Models. In *Proceedings of Conference on Machine Learning and Systems (MLSys)*, 2021.
 - [65] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, Graham Cormode, and Ilya Mironov. Opacus: User-Friendly Differential Privacy Library in PyTorch. In *arxiv.org*, 2021.
 - [66] Da Yu, Saurabh Naik, Arturs Backurs, Sivakanth Gopi, Huseyin A Inan, Gautam Kamath, Janardhan Kulkarni, Yin Tat Lee, Andre Manoel, Lukas Wutschitz, Sergey Yekhanin, and Huishuai Zhang. Differentially Private Fine-tuning of Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
 - [67] Weijie Zhao, Jingyuan Zhang, Deping Xie, Yulei Qian, Ronglai Jia, and Ping Li. AIBox: CTR Prediction Model Training on a Single Node. In *Proceedings of the ACM International Conference on Information and Knowledge Management*, 2019.